

Contents

SVM Compute Core — Full-Chip Design Summary (m5/v10: Final Harden)	1
Component Summary	1
Functional Results	2
Training Set	2
Testing Set	3
Batch Architecture (v10)	3
Wishbone Register Map (base 0x3000_0000)	4
Full-Chip Power Estimate	5
Caravel Submission Artifacts	5
Acknowledgments	6
Feature Extraction References	6
Appendix A — Runtime Model Reload Procedure	7
Appendix B — Alternative Design: Hospital-Grade Batch Classifier (28nm)	9
Appendix C — MCU Task Sequence	13
Appendix B.10 — RTL Improvements	15
Appendix B.11 — Model Improvements for Next Iteration	16
Appendix B.12 — System-Level Improvements for Next Iteration	16

SVM Compute Core — Full-Chip Design Summary (m5/v10: Final Harden)

Project: Multi-Class Cardiac Arrhythmia Detection — Caravel chipIgnite Tape-Out **Tech-**
nology: sky130A / sky130_fd_sc_hd **Flow:** OpenLane 2 v2.3.10 Classic **Architecture:**
Batch v10 — host pre-loads SV + input matrix; ASIC classifies autonomously **RTL freeze:**
m5/rt1 v10 — NUM_SV=500, alpha_addr[8:0], reg_alpha_wr[24:0] **Harden version:** v10 —
RAM_LATENCY=3 (IS61WV51216 SRAM), RUN_MCSTA=1 (SS/FF/TT corner signoff)

Component Summary

svm_compute_core (job 92840, v10)

Metric	Value
Clock	40 MHz (25 ns)
Setup WNS — TT (nom_tt_025C_1v80)	+3.96 ns — 0 violations [OK]
Setup WNS — FF (nom_ff_n40C_1v95)	+11.24 ns — 0 violations [OK]
Setup WNS — SS (nom_ss_100C_1v60)	-14.56 ns — 163 violations [WARN] (100 degC/1.60V, expected for sky130)
Hold WNS — TT	+0.23 ns — 0 violations [OK]
Hold WNS — SS / FF	+0.70 ns / +0.06 ns — 0 violations [OK]
DRC	0 violations [OK]
LVS	PASSED [OK]
Antenna	554 net / 808 pin violations [WARN] (advisory; DRC clean)

Metric	Value
Active power (TT)	55.25 mW (37.2% clock, 46.4% seq, 16.4% comb)
Inference time	9.87 ms / beat (LAT=3, IS61WV51216 SRAM); 3.23 ms (LAT=1 cosim)
Avg power (80 bpm)	0.869 mW (1.316% duty cycle, LAT=3)
Die area	2500 x 2500 umm (15.0% utilization, 157,991 std cells)
GDS	226 MB
LEF	94 KB
GL netlist	13 MB

user__project__wrapper (job 92861, v10)

Metric	Value
Die area	2920 x 3520 umm (Caravel fixed, FP_DEF_TEMPLATE)
Macro	u_svm at (253, 554) N — 2500 x 2500 umm footprint
Clock	wb_clk_i (Caravel), gated to svm_gclk via ICG
CTS	Disabled (RUN_CTS: 0)
DRC	11,923 Magic DRC (boundary artifacts — acceptable)
LVS	1,683 errors (boundary artifacts — acceptable)
GDS	230 MB
LEF	195 KB
GL netlist	78 KB

Functional Results

Implementation	Accuracy	SVs	Notes
sklearn OVR (float)	97.67%	416 total (unlimited)	float precision
ASIC binary OVR (Q6.10)	97.67%	500 total (100x5)	gamma=0.25, C=1.0

Zero accuracy gap — ASIC exactly matches sklearn on all 300 test samples. See Testing Set section for per-class breakdown.

Training Set

Parameter	Value
Source	MIT-BIH + SVDB + INCART (PhysioNet) — pooled, stratified
Split	80% train / 20% test — stratified, <code>random_state=42</code>
Beats per class	240
Classes	Normal (N), PVC, AFib, VT, SVT
Total training beats	1,200
Feature dimension	256 (128 single-beat + 64 10-beat mean + 64 RR history)
Precision	Q6.10 fixed-point (16-bit signed)

Testing Set

Parameter	Value
Source	MIT-BIH + SVDB + INCART (PhysioNet) — held-out 20% stratified
Beats per class	60
Total test beats	300
sklearn accuracy	97.67% (293/300)
ASIC accuracy	97.67% (293/300) — zero gap vs. float

Per-class test results:

Class	Correct	Accuracy
Normal (N)	60/60	100.0%
PVC	60/60	100.0%
AFib	60/60	100.0%
VT	56/60	93.3%
SVT	57/60	95.0%

Batch Architecture (v10)

Off-chip RAM Bus

Signal	Pin	Direction	Description
<code>ram_addr[18:0]</code>	GPIO[28:10]	ASIC out	{row[10:0], col[7:0]}
<code>ram_ren</code>	GPIO[29]	ASIC out	Read strobe

Signal	Pin	Direction	Description
ram_rdata[15:0]	LA[15:0]	Host in->ASIC	Data valid after RAM_LATENCY cycles

Address layout: rows 0..499 = SV matrix; rows 500..1499 = input matrix.

RAM_LATENCY parameter — configures the number of clock cycles between `ram_ren` assertion and valid `ram_rdata`. Default is 1 (same-cycle combinational SRAM model used in cosim). Set to 3 for the IS61WV51216 async SRAM (10 ns access at 40 MHz -> 3-cycle pipeline). The core inserts wait states automatically; the host does not need to pad responses. Verified by `svm_ram_latency_tb.sv` (10-beat, LAT=3, PASS in ~208 cycles/beat).

Rationale for LAT=3 over LAT=2: The IS61WV51216 specifies 10 ns access time, which fits within one 25 ns clock cycle in theory. However LAT=3 is used to provide a safety margin for: (1) PCB trace propagation delays between the ASIC GPIO pins and the SRAM data pins, (2) input flip-flop setup time inside the ASIC, and (3) SRAM slowdown at elevated temperature and low voltage. LAT=2 may work on a bench at room temperature but risks intermittent failures in field conditions. The throughput penalty (9.7 ms vs 3.23 ms per beat) is acceptable given the 750 ms heartbeat window at 80 bpm.

What the Host Does

MCU (low-power, continuous)

```
|
| 1. Collect 1000 heartbeats (250 Hz ECG -> feature extraction)
| 2. Load SV matrix (500 SVs x 256 features) -> SRAM rows 0..499
| 3. Load input matrix (1000 beats x 256 features) -> SRAM rows 500..1499
| 4. Write NUM_SAMPLES = 1000, write NUM_SV_0--4 = 100 each
| 5. Write alpha coefficients via ALPHA_WR (Wishbone 0x28)
| 6. Fire CONTROL[start]
|
v ASIC takes over (timing shown for RAM_LATENCY=1 / RAM_LATENCY=3):
+-- LOAD_INPUT per beat: 256 / 768 cycles (reads from SRAM rows 500+)
+-- COMPUTE_DIST per SV: 258 / 770 cycles (reads SV from SRAM rows 0--499)
+-- COMPUTE_KERNEL: ~18 cycles (Horner LUT exp approximation, LAT-independent)
+-- WRITE_CLASS: argmax+alpha -> sample_rdy (IRQ[0]) per beat
      last beat -> done (IRQ[1])
```

Inference time scales linearly with RAM_LATENCY. At 40 MHz: - LAT=1 (cosim default): **3.23 ms / beat** (500 SVs x 256 dim) - LAT=3 (IS61WV51216 SRAM): **~9.7 ms / beat** — still well within the 750 ms heartbeat period

Wishbone Register Map (base 0x3000_0000)

Offset	Name	R/W	Description
+0x04	CONTROL	RW	[0]=start [1]=vbatt_ok [2]=vbatt_warn
+0x08	STATUS	RO	[0]=done [1]=error [5:2]=error_code [8:6]=class [9]=sample_rdy
+0x0C	NUM_SAMPLES	RW	[9:0] beats in batch (1–1000)
+0x10–+0x20	NUM_SV[0–4]	RW	[7:0] SVs per class (max 100 each)
+0x24	PARAM_WR	WO	[19]=en [18:16]=addr [15:0]=data (gamma, C, bias)
+0x28	ALPHA_WR	WO	[24:16]=sv_global_idx (9-bit) [15:0]=alpha Q6.10

Full-Chip Power Estimate

Subsystem	Active Power	Duty Cycle	Avg Power
svm_compute_core (batch)	66 mW	1.316% (9.87 ms / 750 ms, LAT=3)	0.869 mW
Caravel management SoC	~5 mW	~5%	~0.25 mW
ECG frontend (analog)	~0.5 mW	100%	0.5 mW
BLE (optional, logging)	~10 mW	~0.1%	~0.01 mW
Total estimated	—	—	~1.63 mW

Battery budget: 200 mAh @ 3.7V = 740 mWh -> **740 mWh / 1.63 mW ~ 454 hours (~18.9 days)**. 14-day target met with 1.35x margin. SVM core alone: ~851 hours (~35.4 days).

Caravel Submission Artifacts

Repository: github.com/adamleehandwerger/caravel_svm_project

GDS Release: v2.0-hardened (226 MB core / 230 MB wrapper)

Layout artifacts:

File	Size	Job	Status
gds/svm_compute_core.gds	226 MB	92840	[OK]
gds/user_project_wrapper.gds	230 MB	92861	pending
lef/svm_compute_core.lef	94 KB	92840	[OK]
lef/user_project_wrapper.lef	195 KB	92861	pending

Verilog artifacts:

File	Size	Job	Status
verilog/gl/svm_compute_core.v	13 MB	92840	[OK]
verilog/gl/user_project_wrapper.v	78 KB	92861	pending
verilog/rtl/svm_compute_core.sv	—	v10	[OK]
verilog/rtl/user_project_wrapper.sv	—	v10	[OK]

Acknowledgments

Place-and-route was performed on **Orca**, Portland State University’s high-performance computing cluster, using SLURM batch jobs with OpenLane 2 v2.3.10 inside a Singularity container. We thank the PSU Research Computing team for providing access to the GPU and CPU nodes that made the multi-hour OpenLane runs feasible.

Feature Extraction References

The 256-dim multi-scale feature vector follows established AAMI EC57 beat classification conventions:

Feature group	Dims	Reference
Single-beat morphology (+/-64 samples, amplitude-norm)	128	de Chazal P, O’Dwyer M, Reilly RB. “Automatic classification of heartbeats using ECG morphology and heartbeat interval features.” <i>IEEE Trans Biomed Eng</i> 51(7):1196-206, 2004. DOI: 10.1109/TBME.2004.827359
10-beat mean morphology template	64	de Chazal P, Reilly RB. “A patient-adapting heartbeat classifier using ECG morphology and heartbeat interval features.” <i>IEEE Trans Biomed Eng</i> 53(12):2535-43, 2006. DOI: 10.1109/TBME.2006.883802
RR-interval history (99 intervals -> 64 pts, norm to NORMAL_RR=308 ms)	64	Llamedo M, Martinez JP. “Heartbeat classification using feature selection driven by database generalization criteria.” <i>IEEE Trans Biomed Eng</i> 58(3):616-25, 2011. DOI: 10.1109/TBME.2010.2068048

Standard: AAMI ANSI EC57:2012 — Performance Requirements for Ambulatory ECG Analysers.
 Datasets: PhysioNet MIT-BIH Arrhythmia Database (Moody GB, Mark RG, 2001) DOI: 10.13026/C2F305;
 SVDB (Greenwald SD, 1990); INCART (Taddei A et al., 1992) — all via wfdb Python package.

Document version: m5/v10 . 2026-06-06 — RAM_LATENCY=3 (IS61WV51216 SRAM); RUN_MCSTA=1 (SS/FF/TT corner signoff); harden jobs 92840/92861

Appendix A — Runtime Model Reload Procedure

The ASIC is fully runtime-reprogrammable. A new trained SVM model can be loaded without resetting the chip or re-synthesizing. The on-chip alpha table and the off-chip SRAM are simply overwritten in place.

What constitutes a “model”

Parameter	Location	Size
SV matrix (500 x 256 x 16-bit features)	Off-chip SRAM, rows 0–499	256 KB
Alpha coefficients (500 x 16-bit)	On-chip <code>alpha_table[]</code> registers	8 KB
Gamma (gamma, Q6.10)	On-chip <code>gamma_reg</code> (PARAM_WR addr=0)	2 bytes
C (regularization, Q6.10)	On-chip <code>c_int</code> (PARAM_WR addr=1)	2 bytes
Bias[0–4] (per-class, Q6.10)	On-chip <code>bias_int[0--4]</code> (PARAM_WR addr=2–6)	10 bytes
SV counts per class (NUM_SV[0--4])	On-chip Wishbone registers	5 bytes

Reload sequence

Perform these steps while the ASIC is idle (STATUS[done]=1 or after reset). Do **not** fire CONTROL[start] until all steps are complete.

Step 1 — Write new SV matrix to off-chip SRAM

Write 500 rows x 256 columns of Q6.10 feature values to SRAM address rows 0–499. Address encoding: `addr[18:0] = {sv_global_idx[10:0], feat_dim[7:0]}`. This is a bulk SRAM write from the MCU and does not involve the Wishbone bus.

```
for sv in range(500):
    for feat in range(256):
        sram_write(addr=(sv << 8) | feat, data=sv_matrix[sv][feat])
```

Step 2 — Write new alpha coefficients via ALPHA_WR

Write all 500 alpha values over Wishbone. Each write encodes the SV global index and the alpha value in a single 32-bit word:

```
WB base = 0x3000_0000
ALPHA_WR offset = 0x28
```

```
for idx in range(500):
    word = (idx << 16) | (alpha[idx] & 0xFFFF)    # [24:16]=sv_idx, [15:0]=alpha Q6.10
    wishbone_write(WB_BASE + 0x28, word)
```

Step 3 — Update gamma, C, and biases via PARAM_WR

PARAM_WR (0x24): [19]=en, [18:16]=addr, [15:0]=Q6.10 value

addr	Parameter
0	Gamma (gamma)
1	C (regularization)
2–6	Bias[0–4] (per-class)

```
PARAM_WR offset = 0x24
```

```
def param_write(addr, val_Q6_10):
    wishbone_write(WB_BASE + 0x24, (1 << 19) | (addr << 16) | (val_Q6_10 & 0xFFFF))

param_write(0, gamma_Q6_10)
param_write(1, C_Q6_10)
for c in range(5):
    param_write(2 + c, bias_Q6_10[c])
```

Step 4 — Update SV counts if changed

Only required if the new model has different SVs per class than the previous model.

```
NUM_SV offsets: class 0 = 0x10, class 1 = 0x14, ..., class 4 = 0x20
for c in range(5):
    wishbone_write(WB_BASE + 0x10 + c*4, num_sv[c])
```

Step 5 — Fire start

```
wishbone_write(WB_BASE + 0x04, 0x0B)    # CONTROL: start=1, vbatt_ok=1, kern_ready=1
```

Constraints

- Total SV count must not exceed 500 (100 per class maximum). Models exceeding this require re-synthesis with updated NUM_SV parameters.
- Gamma must be representable in Q6.10 (range 0–63.999, resolution ~0.001).
- Alpha values must be representable in Q6.10 (signed, range -32 to +31.999).
- The reload can be performed between any two batches. The ASIC does not need to be held in reset during the write sequence — the alpha table and gamma register are shadow-registered and only take effect when **start** fires.
- Writing ALPHA_WR while STATUS[done]=0 (i.e., during active inference) has undefined behavior and must be avoided.

Appendix B — Alternative Design: Hospital-Grade Batch Classifier (28nm)

This appendix specifies an alternative architecture targeting continuous bedside cardiac monitoring in a hospital environment — no power or area constraints, 100-beat batch processing, GEMM-based matrix engine.

B.1 Design Goals

Parameter	Wearable (current)	Hospital (proposed)
Use case	Ambulatory patch	Bedside monitor / ICU
Batch size	1000 beats	100 beats
Latency requirement	< 750 ms/beat	< 5 s per 100-beat batch
Power budget	< 1 mW avg	No constraint
Area budget	Caravel die (6.25 mm ²)	No constraint
Process	sky130A (180nm equiv.)	TSMC 28nm HPC

B.2 Feature Set (unchanged)

The same 256-dimensional multi-scale feature vector is used:

Group	Dims	Content
Single-beat morphology	128	+/-64 samples, amplitude-normalized
10-beat mean template	64	Average of preceding 10 beats
100-beat mean template	64	Average of preceding 100 beats (replaces RR-interval history)
Total	256	

The 100-beat template replaces the RR-interval history for hospital use, providing longer-term morphological context available in a continuous-monitoring environment.

B.3 Architecture

Compute engine: 32x32 systolic array

A 32x32 weight-stationary systolic array processes the pairwise distance matrix as a single GEMM operation rather than sequentially iterating over SVs.

The distance computation is restructured as:

$$\begin{aligned} D[i,j] &= ||X[i] - SV[j]||^2 \\ &= ||X[i]||^2 - 2.(X \cdot SV^T)[i,j] + ||SV[j]||^2 \end{aligned}$$

The dominant term $X \cdot SV^T$ is a **100x256 x 256x500** GEMM — 12.8M MACs, fully parallelized across the systolic array. The squared norm terms are precomputed in a single pass and broadcast.

Memory: on-chip SRAM (1 MB total)

Buffer	Size	Contents
Input matrix	51.2 KB	100 beats x 256 features x 16-bit Q6.10
SV matrix	256 KB	500 SVs x 256 features x 16-bit Q6.10
Distance matrix	200 KB	100 x 500 x 32-bit intermediate
Kernel matrix	100 KB	100 x 500 x 16-bit exp output
Alpha table	1 KB	500 x 16-bit (same as current)
Score / output	2 KB	100 x 5 x 32-bit class accumulators
Total	~610 KB	(rounded to 1 MB with ECC and overhead)

Clock and process

Parameter	Value
Process	TSMC 28nm HPC
Supply voltage	0.9 V
Clock	800 MHz
Peak compute	1024 MACs/cycle x 800 MHz = 819 GOPS
On-chip SRAM bandwidth	~100 GB/s

B.4 Performance

Per 100-beat batch:

Stage	Operations	Cycles	Time
Load input + SV to SRAM	—	—	one-time, DMA
GEMM: $X \cdot SV^T$ (100x256x500)	12.8M MACs	~12,500	15.6 ums
Squared norms + broadcast	100K ops	~100	0.1 ums
Exp LUT (100x500 evaluations)	50K	50,000	62.5 ums
Alpha accumulation (100x500x5)	250K MACs	~250	0.3 ums
Argmax (100 x 5)	500	~1	<0.1 ums
Total per 100-beat batch		~63,000	~78 ums

Throughput: 100 beats / 78 ums = **1,280,000 inf/s (1.28 M inf/s)**

Duty cycle at 80 bpm (100 beats every 75 s): 78 ums / 75,000,000 ums = **0.000104%**

B.5 Power

Component	Active	Standby
Systolic array (32x32, 0.9V, 800 MHz)	~180 mW	~0

Component	Active	Standby
On-chip SRAM (1 MB active)	~80 mW	~3 mW leakage
Control logic, IO	~40 mW	~0.5 mW
Total	~300 mW	~3.5 mW

Average power at 80 bpm (100-beat batches):

$$\begin{aligned}
P_{\text{avg}} &= P_{\text{active}} \times \text{duty_cycle} + P_{\text{leakage}} \\
&= 300 \text{ mW} \times 0.000104\% + 3.5 \text{ mW} \\
&\sim 0.0003 \text{ mW} + 3.5 \text{ mW} \\
&= \sim 3.5 \text{ mW}
\end{aligned}$$

Average power is dominated entirely by SRAM leakage. The compute contribution is negligible — the chip spends 99.9999% of the time idle.

B.6 Die Area

Block	Area (28nm est.)
1 MB on-chip SRAM	~0.50 mm ²
32x32 systolic array	~0.30 mm ²
Horner LUT + exp pipeline	~0.05 mm ²
Control FSM, IO, misc	~0.10 mm ²
Total estimated	~1.0 mm²

The hospital design is **6x smaller die area** than the current sky130A core (6.25 mm²) despite being orders of magnitude more capable — a direct consequence of the 28nm process density advantage.

B.7 Roofline Comparison

The fundamental shift from the wearable to the hospital design is the operational intensity: the GEMM formulation reuses the SV matrix across all 100 input beats, dramatically increasing arithmetic intensity.

Operational intensity:

Design	Total ops	Memory traffic	Ops/byte
Wearable (sequential, per beat)	512K ops	256 KB (SV re-read each beat)	2.0
Hospital (GEMM, per 100-beat batch)	25.6M ops	307 KB (SV loaded once)	~83

The wearable design sits deep in the **memory-bound** region of the roofline — 98.9% of time waiting for SRAM data. The hospital design operates well above the ridge point (~16 ops/byte at 100 GB/s / 1.6 TOPS) and is firmly **compute-bound**.

B.8 Comparison Table

Throughput and latency:

Metric	Wearable ASIC	Hospital ASIC	sklearn	Numba (8-core)
Throughput	101 inf/s (LAT=3)	1,280,000 inf/s	4,200 inf/s	95,000 inf/s
Latency / 100 beats	987 ms (LAT=3)	0.078 ms	23.8 ms	1.05 ms
Speedup vs sklearn	0.024x	305x	1x	22.6x

Power, area and roofline:

Metric	Wearable ASIC	Hospital ASIC	sklearn	Numba (8-core)
Active power	66 mW	300 mW	15,000 mW	80,000 mW
Avg power (80 bpm)	0.284 mW	3.5 mW	~15,000 mW	~80,000 mW
Die area	6.25 mm ²	~1.0 mm ²	N/A	N/A
Process	sky130A (180nm)	TSMC 28nm	Intel 10nm	Intel 10nm
Ops/byte	2.0	83	~2.0	~2.0
Roofline regime	Memory-bound	Compute-bound	Memory- bound	Memory-bound

Key observations:

- The hospital ASIC is **305x faster than sklearn** and **13.5x faster than 8-core Numba** on a 100-beat batch
- This is achieved by converting a memory-bound sequential problem into a compute-bound GEMM
- The SRAM leakage floor (3.5 mW) makes the hospital design unsuitable for wearable use — 12x worse average power than the current design despite being 4,000x faster
- Both designs achieve 97.67% accuracy — the architecture difference is entirely in throughput and power, not classification quality

B.9 SRAM Latency in the Hospital Design

The wearable ASIC uses `RAM_LATENCY=3` to interface the IS61WV51216 async SRAM (10 ns access time) at 40 MHz. The 3-cycle wait accounts for PCB trace delay, ASIC input flip-flop setup time, and SRAM derating at elevated temperature.

The hospital design (28nm, 800 MHz, 0.9 V, on-chip SRAM) has no off-chip SRAM interface at all — the SV and input matrices fit in 610 KB of on-chip memory. If the hospital design were instead deployed with an external SRAM (e.g., for a larger model exceeding on-chip capacity), a faster async part such as the **IS61WV102416** (5 ns access) would allow `RAM_LATENCY=2` at 800 MHz (5 ns < 1.25 ns x 2 cycles — marginal; in practice `RAM_LATENCY=3` would still be prudent at 800 MHz). At a more conservative 400 MHz clock (2.5 ns period), a 5 ns SRAM fits cleanly in `RAM_LATENCY=2` with margin for PCB and setup.

The general rule: `RAM_LATENCY = ceil((t_access + t_PCB + t_setup) / t_clk)`. For the wearable at 40 MHz: `ceil((10 + 2 + 1) / 25) = ceil(0.52) = 1` — theoretically `LAT=1` works on a benchtop, but `LAT=3` is used for field margin.

Appendix C — MCU Task Sequence

C.0 — MCU Integration and Prototype Bringup

The term “host” refers to whatever processor issues Wishbone register writes and reads GPIO results. The ASIC is agnostic to which processor plays this role.

On Caravel silicon (tape-out): The on-chip RISC-V management core (PicoRV32) is the host. It runs compiled C firmware (`svm_wb_test.c`), issues all Wishbone writes, and monitors GPIO for `sample_rdy` and `class_out`. The RISC-V is also the host during the Caravel DV simulation (Level 6 testbench).

On a wearable PCB (production): An external low-power MCU (e.g. STM32L4) becomes the host. It connects to the ASIC’s Wishbone interface via SPI or I²C bridge and drives GPIO directly. The RISC-V management core is bypassed or unused.

For prototype bringup when silicon arrives: The chain is PC -> Caravel -> ASIC:

1. Connect a laptop to the Caravel chip via the **housekeeping SPI** port — a dedicated SPI interface on the Caravel die for external firmware loading and GPIO monitoring.
2. Use the Efabless `caravel_board` Python utility to flash `svm_wb_test.c` firmware onto the management core’s on-chip flash.
3. The PicoRV32 boots, runs the firmware, and issues Wishbone writes to configure the SVM (alpha coefficients, SV counts, gamma, `NUM_SAMPLES`).
4. The RISC-V fires `start` via `CONTROL[0]`, monitors `sample_rdy` on GPIO, and echoes results back to the laptop over UART.

In all three cases the ASIC interface is identical — Wishbone registers at `0x3000_0000`, GPIO address bus on `GPIO[28:10]`, read data on `la_data_in[15:0]`. Switching from prototype to production only requires replacing the firmware host; no RTL changes.

The MCU drives every phase of the system. The ASIC is passive until `start` fires and runs autonomously until `done` pulses. The MCU must not assume `start` self-clears — if `start` remains asserted when the FSM returns to IDLE, the ASIC immediately begins another batch on the same data.

Per-batch sequence

Phase 1 — Data loading (MCU active, ASIC idle)

1. Collect 1000 heartbeats via ECG frontend (250 Hz sampling, feature extraction per beat)
2. Write SV matrix to off-chip SRAM: 500 rows x 256 Q6.10 features -> rows 0–499
3. Write input matrix to off-chip SRAM: up to 1000 beats x 256 Q6.10 features -> rows 500–1499
4. Write alpha coefficients via ALPHA_WR (Wishbone 0x28): one write per SV (500 total)
5. Write NUM_SAMPLES (Wishbone 0x0C): number of beats in this batch
6. Write NUM_SV[0–4] (Wishbone 0x10--0x20): SVs per class (100 each)

Phase 2 — Fire and sleep (MCU sleeps, ASIC classifies)

7. Assert **start**: write 0x0B to CONTROL (0x04) — sets **start**=1, **vbatt_ok**=1, **kern_ready**=1
8. MCU enters deep sleep — ASIC classifies the full batch autonomously (~9.87 ms at LAT=3)

Phase 3 — Done handling (MCU wakes on IRQ)

9. IRQ[1] (done) fires — MCU wakes from sleep
10. **Clear start**: write 0x0A to CONTROL (0x04) — clears bit 0, leaving **vbatt_ok** and **kern_ready** set. If **start** is not cleared before the FSM reaches IDLE, the ASIC will immediately re-classify the same batch.
11. Read STATUS (0x08): bits [8:6] = **class_out** for the final beat; bit [1] = error flag
12. Per-beat results were signalled by **sample_rdy** (IRQ[0]) during classification — MCU firmware should have latched **class_out** on each IRQ[0] pulse for the full per-beat result log

Phase 4 — Next batch

13. Load new input matrix into SRAM rows 500–1499 (SV matrix in rows 0–499 is unchanged unless model reloaded)
14. Update NUM_SAMPLES if batch size changed
15. Re-assert **start** -> repeat from Phase 2

Register writes summary

Step	Register	Offset	Value	Effect
Load alpha	ALPHA_WR	0x28	{sv_idx[8:0], alpha[15:0]}	Write one alpha coefficient
Set batch size	NUM_SAMPLES	0x0C	0x03E8 (1000)	Beats per batch
Set SV counts	NUM_SV[0–4]	0x10--0x20	0x64 (100) each	SVs per class
Fire	CONTROL	0x04	0x0B	start=1, vbatt_ok=1, kern_ready=1
Clear	CONTROL	0x04	0x0A	start=0, vbatt_ok=1, kern_ready=1
Read result	STATUS	0x08	—	[8:6]=class, [1]=error, [0]=done

C.2 — Data history storage

Per-beat `class_out` values captured from `IRQ[0]` pulses are held in the nRF52840's 256 KB internal SRAM during classification. For long-term history that must survive power cycles, the MCU flushes each completed batch to external non-volatile storage.

External flash (recommended for wearable logging)

The nRF52840 QSPI interface connects to a NOR flash such as the MX25R6435F (4 MB, 1.8 V, ultra-low standby ~4 uA). Each classification record is compact — 3-bit class + 32-bit timestamp = 5 bytes — so 4 MB holds ~800,000 beat records (~11 days at 80 bpm continuous). The MCU buffers one batch in internal SRAM, then issues a single QSPI page-program on `done` IRQ before sleeping.

BLE streaming (recommended for real-time clinical offload)

The nRF52840 BLE stack can stream `class_out` records to a paired phone or gateway in real time. Each `sample_rdy` IRQ enqueues a 5-byte record into a BLE notification buffer; the phone application handles long-term storage and trend analysis. No external flash is required if connectivity is assumed.

Hybrid (wearable + clinic)

Buffer to flash during normal wear; bulk-transfer history to phone over BLE when within range. The nRF52840 supports concurrent QSPI and BLE without additional hardware.

Appendix B.10 — RTL Improvements

B.10.1 NUM_SAMPLES reset default

`reg_num_samples` resets to `10'd0` on power-on. Unlike `NUM_SV[0--4]` which reset to `8'd50` (a safe non-zero default), a zero `NUM_SAMPLES` causes the batch FSM to complete immediately with no classifications if the MCU forgets to write the register before asserting `start` — a silent failure with no error flag.

Recommended fix: change the reset assignment in `top.sv`:

```
// current
reg_num_samples <= 0;

// improved
reg_num_samples <= 10'd1000; // match nominal deployment batch size
```

This makes `NUM_SAMPLES` consistent with `NUM_SV` (sticky, sensible default) and eliminates a bringup gotcha where a forgotten register write produces a zero-beat batch that appears to succeed.

B.10.2 NUM_SAMPLES sticky behavior

`NUM_SAMPLES` should retain its value across batches without being rewritten by the MCU each time. Currently the register holds its value (Wishbone registers are persistent by design), but the reset default of `10'd0` means the first batch after power-on will silently process zero beats unless the MCU writes the register before firing `start`.

Recommended fix: in addition to the reset default change in B.10.1, document `NUM_SAMPLES` as a sticky configuration register in firmware — write it once at startup alongside `NUM_SV[0--4]`, `ALPHA_WR`, and `PARAM_WR`, and only rewrite if the batch size changes. This matches the behavior of all other configuration registers and avoids a class of firmware bugs where the register is written in the first batch but forgotten in subsequent batches after a partial reset.

Appendix B.11 — Model Improvements for Next Iteration

B.11.1 Class weight tuning

The current model uses `class_weight='balanced'` in sklearn, which scales the SVM penalty parameter C inversely with class frequency. This corrects for the MIT-BIH class imbalance (Normal beats dominate) but does not account for **clinical asymmetry** — missing a VT is far more dangerous than missing a PVC or SVT.

`class_weight='balanced'` treats all arrhythmia classes equally after frequency correction. For clinical deployment, a better approach is to treat per-class weights as hyperparameters and tune them explicitly, optimizing for **recall on VT and AFib** subject to a constraint on the Normal false-positive rate rather than optimizing for overall accuracy.

A grid search over class weights (e.g. boosting VT weight 2–4x beyond balanced) would likely maintain or improve the 97.67% overall accuracy while providing stronger guarantees on the clinically dangerous classes. The zero-gap property would need to be re-verified at Q6.10 after retraining with new weights.

Impact on hardware: class weights only affect training. The ASIC inference path (kernel evaluation, alpha accumulation, argmax) is unchanged — new weights produce new alpha coefficients and bias values loaded via the Wishbone parameter interface at startup.

Appendix B.12 — System-Level Improvements for Next Iteration

B.12.1 Argmax confidence and OvR score calibration

The current hardware outputs a hard class label via argmax across 5 OvR scores with no confidence information. This creates two clinical risks:

Out-of-distribution beats: If the input beat is unlike any training data (e.g., pacemaker spikes, WPW, artifact), all 5 scores may be low and the argmax picks the least-bad class with no indication of low confidence. The device outputs a definitive label when it should flag the beat for physician review.

OvR score miscalibration: The 5 binary classifiers are trained independently with no guarantee their score scales are comparable. A classifier trained on a highly imbalanced class may produce scores with systematically larger or smaller magnitude than others. Argmax across uncalibrated scores can favor a class not because it is most likely but because its scores are larger in magnitude.

Recommended improvements:

1. **Platt scaling** — fit a sigmoid to each classifier’s raw scores post-training to convert them to calibrated probabilities. Argmax over probabilities is better justified than argmax over raw margins.

2. **Confidence threshold** — add a **confidence** output to STATUS register encoding the margin between the top two scores. If the margin is below a threshold, assert a **low_confidence** flag so firmware can defer to a clinician rather than acting on an uncertain classification.
3. **Unknown class** — add a 6th “unknown/artifact” class trained on out-of-distribution beats (pacemaker, noise, WPW). This requires an additional classifier and alpha table entry but eliminates silent misclassification of unseen beat types.

Impact on hardware: Platt scaling only changes the bias and alpha values (training change). The confidence threshold requires one additional STATUS register bit and a comparator on the two highest accumulators — minimal RTL change. The unknown class requires widening **class_out** from 3 to 3 bits (still fits) and adding one more OvR classifier (6th alpha table partition of 100 SVs).